

Segundo Trabajo Grupal

Programación 1 - Sistema de Gestión Bibliotecario

Objetivo

La biblioteca de la UCU necesita un sistema para llevar un control de sus libros, usuarios y préstamos. Se les pide implementar dicho sistema de forma que permita registrar qué libros existen y quién los tiene prestados, manteniendo toda la información persistida en archivos. Deberán implementarlo mediante el uso de **listas, diccionarios, archivos y manejo de excepciones**. No se requiere ningún concepto adicional para resolver este trabajo.

Descripción del sistema

El sistema manejará tres entidades principales, representadas mediante diccionarios:

Libro

Cada libro se representa como un diccionario con las siguientes claves:

- **isbn**: string único que identifica al libro.
- **titulo**: string con el título del libro.
- **autor**: string con el autor del libro.
- **genero**: string con el género del libro.
- **ejemplares_totales**: entero que indica la cantidad total de ejemplares
- **ejemplares_disponibles**: enteros que indican la cantidad de ejemplares disponibles para préstamo.

Ejemplo

```
{ "isbn": "978-1", "titulo": "Cien años de soledad",  
  "autor": "García Márquez", "genero": "Novela",  
  "ejemplares_totales": 2, "ejemplares_disponibles": 2 }
```

Usuario

Cada usuario se representa como un diccionario con las siguientes claves:

- **numero_socio**: entero que identifica al usuario.
- **nombre**: string con el nombre completo.
- **prestamos_activos**: lista de ISBNs de libros prestados actualmente al usuario.

Ejemplo:

```
{ "numero_socio": 1, "nombre": "Ana García",
  "prestamos_activos": ["978-1"] }
```

Préstamo

Cada préstamo se representa como un diccionario con las siguientes claves:

- **numero_socio**: identifica al usuario que realizó el préstamo.
- **isbn**: identifica el libro prestado.
- **fecha_prestamo**: string que representa la fecha del préstamo.
- **fecha_limite**: string que representa la fecha máxima de devolución del libro. Es la fecha_prestamo más 7 días.
- **devuelto**: booleano que indica si el libro ya fue devuelto.

Ejemplo:

```
{ "numero_socio": 1, "isbn": "978-1",
  "fecha_prestamo": "02/06/2026",
  "fecha_limite": "09/06/2026", "devuelto": False }
```

Estructuras globales

El estado completo del sistema se mantiene en tres estructuras, que se pasan como parámetro en cada función:

- **catalogo**: diccionario cuya clave es el ISBN y cuyo valor es el diccionario del libro.
- **usuarios**: diccionario cuya clave es el número de socio y cuyo valor es el diccionario del usuario.
- **prestamos**: lista de diccionarios, donde cada uno corresponde a un préstamo registrado.

Observación: toda operación que modifique el estado del sistema debe quedar persistida en el archivo correspondiente.

Funciones a implementar

Función	Descripción
---------	-------------

agregar_libro(catalogo, isbn, titulo, autor, genero, ejemplares)	Agrega un libro al catálogo. Lanza una excepción si el ISBN ya existe.
eliminar_libro(catalogo, prestamos, isbn)	Elimina un libro por ISBN. Lanza una excepción si no existe o tiene préstamos activos.
buscar_libros(catalogo, termino)	Retorna una lista de libros cuyo título, autor o género contengan el término.
registrar_usuario(usuarios, numero_socio, nombre)	Registra un nuevo usuario. Lanza una excepción si el número de socio ya existe.
dar_baja_usuario(usuarios, prestamos, numero_socio)	Elimina un usuario. Lanza una excepción si tiene préstamos activos.
historial_usuario(prestamos, numero_socio)	Retorna la lista de todos los préstamos (activos y finalizados) de un usuario.
registrar_prestamo(catalogo, usuarios, prestamos, numero_socio, isbn, fecha_prestamo)	Registra un préstamo. Lanza una excepción si el libro no tiene ejemplares o el usuario no existe.
registrar_devolucion(catalogo, prestamos, numero_socio, isbn)	Registra la devolución de un libro y actualiza los ejemplares disponibles.
listar_vencidos(prestamos, fecha_actual)	Retorna una lista de préstamos cuya fecha límite ya pasó.
guardar_datos(catalogo, usuarios, prestamos, ruta)	Guarda el estado del sistema en un archivo.
cargar_datos(ruta)	Carga los datos desde un archivo. Lanza una excepción si no existe.
normalizar(texto)	Convierte el texto a minúsculas y elimina tildes.

Ejemplo paso a paso

Dado el siguiente estado inicial:

```
catalogo = {  
  "978-1": { "isbn": "978-1",  
            "titulo": "Cien años de soledad",  
            "autor": "García Márquez", "genero": "Novela",  
            "ejemplares_totales": 2,  
            "ejemplares_disponibles": 2 },  
  "978-2": { "isbn": "978-2",  
            "titulo": "El túnel",  
            "autor": "Sábato", "genero": "Novela",  
            "ejemplares_totales": 1,  
            "ejemplares_disponibles": 1 }  
}  
usuarios = { "001": { "numero_socio": 001, "nombre": "Ana",  
  "prestamos_activos": [] }  
}  
prestamos = []
```

Al registrar un préstamo mediante la siguiente función:

```
registrar_prestamo(catalogo, usuarios, prestamos, 1, "978-1",  
  "02/06/2026")
```

El nuevo estado se vería de la siguiente forma:

```
catalogo = {
    "978-1": { "isbn": "978-1",
               "titulo": "Cien años de soledad",
               "autor": "García Márquez", "genero": "Novela",
               "ejemplares_totales": 2,
               "ejemplares_disponibles": 2 },
    "978-2": { "isbn": "978-2",
               "titulo": "El túnel",
               "autor": "Sábato", "genero": "Novela",
               "ejemplares_totales": 1,
               "ejemplares_disponibles": 1 }
}

usuarios = { "001": { "numero_socio": 001, "nombre": "Ana",
                     "prestamos_activos": ["978-1"] }
}

prestamos = [{ "numero_socio": 1, "isbn": "978-1",
               "fecha_prestamo": "02/06/2026",
               "fecha_limite": "09/06/2026", "devuelto": False }
]
```

Bonus (opcional)

Dado un número de socio, el sistema debe analizar el historial de préstamos del usuario, identificar los géneros que más ha leído y retornar una lista de libros disponibles que sean de esos géneros y que no los haya tomado prestado anteriormente.

La función debe llamarse **recomendar_libros(catalogo, prestamos, numero_socio, n)** y retornar una lista con hasta n libros recomendados, ordenados por ejemplares disponibles de mayor a menor.

Documentación requerida

La entrega debe incluir un documento PDF con las siguientes secciones:

1. **Análisis del problema y posibles soluciones alternativas.** Explicar con sus palabras qué resuelve el trabajo y qué otras soluciones podrían haberse

utilizado. Especificar qué tareas fueron asignadas a cada integrante del equipo.

2. **Diseño, descripción y justificación de la solución propuesta.** Justificar las decisiones de implementación (estructura de datos elegida para representar libros, usuarios y préstamos; formato del archivo de persistencia, etc.). Explicar el funcionamiento de cada función implementada.
3. **Manual de usuario.** Descripción de los pasos que el usuario del programa debe seguir para operarlo, mostrando capturas de pantalla de cada operación y sus resultados.
4. **Conclusiones del trabajo.** Expresar cuál fue la experiencia al realizar el trabajo, qué aprendizajes aportó y qué cambiarían o agregarían para mejorarlo.

Entrega

Se deberá entregar un archivo comprimido en formato .zip que incluya:

- El código Python completo debidamente comentado.
- Un archivo .pdf con la documentación solicitada (secciones 1, 2 y 4).
- Un archivo .pdf con el manual de usuario (sección 3).
- Un archivo Python con los casos de prueba utilizados para verificar el programa.

Observaciones: el código fuente deberá estar comentado. Solo deben utilizarse los temas vistos en clase: listas, diccionarios, archivos y manejo de excepciones.

Fecha límite de entrega: 26/06/2026 a las 23:59

NO SE ADMITIRÁN ENTREGAS TARDÍAS.